

Dossier technique — InsaCloud

Synthèse du cours, énoncé du projet 3 et analyse de conformité

Mohamed Aziz BAOUEB • STI 4A FISE • 2026/2027

Outils de déploiement de plateformes — Dr. Nadia FETTAH

Ce document accompagne le rapport officiel (limité à 8 pages par les modalités de validation). Il rassemble la synthèse des supports de cours, l'énoncé complet du projet 3 et une analyse de conformité exhaustive, élément par élément.

Table des matières

Partie I — Synthèse des supports de cours	3
1 Virtualisation, conteneurisation et Vagrant	3
1.1 Du serveur physique à l'infrastructure virtualisée	3
1.2 Machines virtuelles et hyperviseurs	3
1.3 Conteneurs	3
1.4 Docker	3
1.5 Docker Compose	4
1.6 Configuration des ressources	4
1.7 Vagrant	5
2 Gestion de configuration avec Ansible	5
2.1 Pourquoi la gestion de configuration	5
2.2 Architecture	5
2.3 Inventaire, commandes ad hoc et playbooks	6
2.4 Idempotence et modules	6
2.5 Variables, facts, templates et handlers	6
2.6 Rôles, Vault, tags et erreurs	6
2.7 Collections, performance et sécurité	7
2.8 Étude de cas : architecture trois tiers	7
Partie II — L'énoncé du projet 3 et les attentes	8
3 Énoncé et besoin à couvrir	8
4 Modalités de validation	8
Partie III — Ce qui a été réalisé et analyse de conformité	9
5 Conformité aux exigences de l'énoncé	9
6 Couverture des notions du cours	10
7 Éléments réalisés au-delà de l'énoncé	12
8 Conformité au document « PROJET 3 » (17 fonctionnalités)	12

9 Écarts relevés et travaux réalisés	14
Conclusion	15

Partie I — Synthèse des supports de cours

1 Virtualisation, conteneurisation et Vagrant

1.1 Du serveur physique à l'infrastructure virtualisée

Le modèle historique « un serveur, une application » souffre de quatre défauts : taux d'utilisation des ressources très faible (10 à 15%), coûts matériels et énergétiques élevés, souplesse quasi nulle (délais d'approvisionnement) et risque opérationnel direct (une panne interrompt le service).

Virtualisation : technique permettant de faire fonctionner plusieurs systèmes logiques sur une même machine physique, isolés les uns des autres, tout en partageant les ressources matérielles.

Le cours distingue cinq formes : matérielle (hyperviseur), système d'exploitation (conteneurs), stockage, réseau (VLAN, overlay, SDN) et applicative. Deux d'entre elles sont étudiées : machines virtuelles et conteneurs.

1.2 Machines virtuelles et hyperviseurs

Une **machine virtuelle** est l'émulation logicielle complète d'un ordinateur : OS invité propre, ressources virtuelles, applications. L'**hyperviseur** (VMM) crée et gère le cycle de vie des VM, alloue les ressources physiques, garantit l'isolation et traduit les instructions privilégiées.

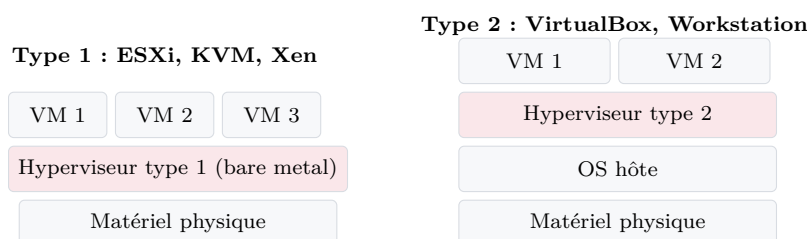


Figure 1 – Les deux familles d'hyperviseurs. Le type 1 s'exécute sur le matériel (production), le type 2 au-dessus d'un OS hôte (développement, tests).

Avantages : isolation forte (noyau propre), OS invités hétérogènes, *snapshots*, clonage, migration à chaud. **Limites** : empreinte disque et mémoire importante, démarrage lent, surcharge de virtualisation, densité limitée. Les extensions processeur Intel VT-x et AMD-V permettent une exécution privilégiée directe, sans traduction logicielle coûteuse.

1.3 Conteneurs

Un **conteneur** est une unité logicielle isolée exécutée par le noyau de l'hôte, sans virtualisation matérielle ni OS invité. Deux mécanismes du noyau Linux le rendent possible :

- **namespaces** — isolent la vue des processus, du réseau, des montages, des utilisateurs (PID, NET, MNT, UTS, IPC, USER, CGROUP) ;
- **cgroups** — limitent, mesurent et isolent la consommation de ressources (CPU, mémoire, E/S, réseau).

1.4 Docker

Docker suit une architecture client-serveur : le client **docker** dialogue avec le démon **dockerd** via une API REST ; **containerd** et **runc** assurent la gestion et l'exécution bas niveau (standard OCI) ; les images proviennent d'un registre (Docker Hub).

Images en couches. Une image est une pile de couches immuables (image de base, dépendances, code, configuration). À l'exécution s'ajoute une couche inscriptible éphémère, gérée en *copy-on-write* :

Critère	Machine virtuelle	Conteneur
Isolation	Matériel/noyau (forte)	Processus (namespaces/cgroups)
Noyau	Propre à chaque VM	Partagé avec l'hôte
Taille type	Plusieurs Go	Quelques Mo à centaines de Mo
Démarrage	Dizaines de secondes à minutes	Centaines de millisecondes
Densité	Faible à moyenne	Élevée
Cas d'usage	Isolation forte, OS hétérogènes	Microservices, CI/CD

Table 1 – Comparaison VM / conteneurs. *Attention* : l'isolation est plus faible qu'une VM, une faille du noyau affecte tous les conteneurs d'un même hôte.

les données y sont perdues à la suppression du conteneur, sauf usage de volumes. Les couches partagées ne sont stockées qu'une fois.

Instructions clés du Dockerfile : FROM (image de base), WORKDIR (répertoire courant), COPY/ADD (copie de fichiers), RUN (commande de construction, nouvelle couche), EXPOSE (documente un port), CMD/ENTRYPOINT (commande de démarrage).

Cycle de vie : build → images → run → ps → logs → exec → stop → rm, complété par pull, start et rmi.

1.5 Docker Compose

Outil de définition et de gestion d'applications multi-conteneurs, décrites dans un fichier YAML `docker-compose.yml` : services, images, ports, réseaux, volumes et dépendances. Exemple du cours (API, base de données, cache) :

```
services:
  api:
    build: ./api
    ports: ["8080:5000"]
    depends_on: [db, cache]
  db:
    image: postgres:16
    volumes: [donnees-db:/var/lib/postgresql/data]
  cache:
    image: redis:7-alpine
volumes:
  donnees-db:
```

1.6 Configuration des ressources

Plusieurs environnements partagent un socle matériel fini. Il faut garantir des performances prévisibles (éviter le « bruyant voisin »), optimiser le taux d'utilisation et empêcher la monopolisation. Trois notions :

- **Réservation** (*request*) : minimum garanti, même en forte contention ;
- **Limite** (*limit*) : plafond maximal ;
- **Priorité relative** (*shares*) : répartition en cas de contention, sans plafond strict.

Règle du cours : toujours définir à la fois une réservation et une limite — ni gaspillage, ni monopolisation. Les contrôleurs cgroups concernés sont `cpu/cpuset`, `memory`, `blkio/io`, `pids` et `net_cls`. Côté Docker :

```
docker run -d --name app-limitee --cpus="1.5" --memory="512m" \
--memory-swap="1g" --memory-reservation="256m" --pids-limit=200 mon-app:1.0
```

Attention à l'OOM Killer : au dépassement de la limite dure, le noyau tue le processus principal. Il faut dimensionner avec une marge. La **supervision** se fait par `docker stats`, `nproc`, `free -h`, `uptime`, `htop`. La **mise à l'échelle** est verticale (*scale up*, plus de ressources) ou horizontale (*scale*

out, plus d'instances). La **surallocation** améliore le taux d'utilisation mais expose au swap et à l'OOM Killer.

1.7 Vagrant

Vagrant (HashiCorp) automatise, à partir d'un fichier déclaratif — le **Vagrantfile** — la création, la configuration et la destruction d'environnements reproductibles. Concepts : **Box** (image de base), **Provider** (moteur de virtualisation), **Provisioner** (Shell, Ansible, Puppet, Chef), **Vagrantfile** (Ruby), **synced folders**.

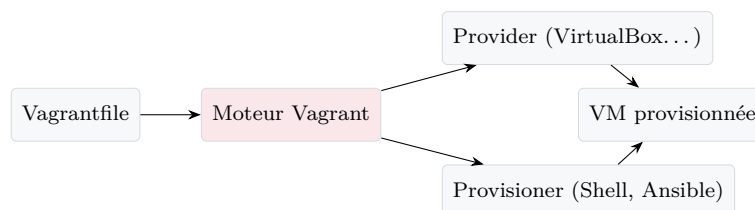


Figure 2 – Architecture générale de Vagrant.

Cycle de vie : init, up, ssh, halt, suspend/resume, reload, destroy. **Bonnes pratiques** : versionner le **Vagrantfile** avec le code, ne pas versionner **.vagrant/**, préférer le provisionnement déclaratif (Ansible) au Shell impératif, fixer la version de la box, combiner VM et Docker selon les besoins.

2 Gestion de configuration avec Ansible

2.1 Pourquoi la gestion de configuration

Configurer manuellement des centaines de serveurs provoque la **dérive de configuration**, la **non-reproductibilité**, l'**absence de traçabilité** et l'. L'IaC décrit l'état désiré sous forme de code déclaratif, versionné et appliqué automatiquement. Deux paradigmes coexistent : **impératif** (suite d'actions) et **déclaratif** (état final souhaité) ; Ansible combine les deux, avec une syntaxe procédurale et des modules idempotents.

Critère	Ansible	Puppet	Chef	SaltStack
Architecture	Sans agent	Agent + maître	Agent + maître	Agent (minion)
Langage	YAML	DSL Ruby	DSL Ruby	YAML / Python
Transport	SSH / WinRM	HTTPS	HTTPS	ZeroMQ / SSH
Apprentissage	Faible	Élevé	Élevé	Moyen
Modèle	Push	Pull	Pull	Push / Pull

Table 2 – Panorama des outils. L'absence d'agent est l'atout majeur d'Ansible.

2.2 Architecture

Le **nœud de contrôle** lit l'inventaire et les playbooks, résout variables et templates Jinja2, transfère les modules Python vers les cibles par SSH, puis collecte les résultats. Les **nœuds gérés** n'hébergent aucun démon : le module est copié, exécuté, puis supprimé.

Flux d'exécution : lecture du playbook et résolution des variables → collecte des *facts* → évaluation des conditions et boucles → transfert du module → exécution et vérification de l'état → retour JSON (ok / changed / failed) et déclenchement éventuel des handlers.

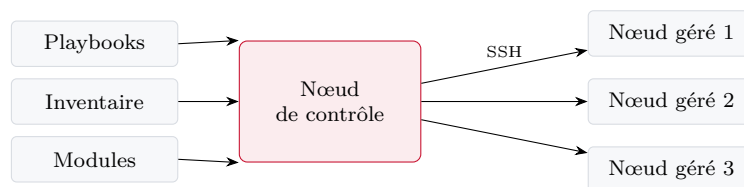


Figure 3 – Architecture d'Ansible : modèle *push*, sans agent permanent sur les cibles.

2.3 Inventaire, commandes ad hoc et playbooks

L'inventaire (INI ou YAML) est la source de vérité : groupes, sous-groupes (`:children`), variables d'hôte et de groupe (`group_vars/`, `host_vars/`). Deux groupes spéciaux existent toujours : `all` et `ungrouped`. Un **inventaire dynamique** interroge une source externe (plugin `amazon.aws.aws_ec2`, `CMDB`).

Une **commande ad hoc** exécute une action ponctuelle sans playbook : `ansible all -i inventaire.ini -m ping`. Un **playbook** contient des *plays*, chaque play associant un groupe d'hôtes à une liste ordonnée de *tasks*. Les tâches s'exécutent séquentiellement, la stratégie par défaut `linear` attend tous les hôtes, les handlers s'exécutent une seule fois en fin de play, et un échec retire l'hôte du reste du play.

2.4 Idempotence et modules

Une opération est **idempotente** si son exécution répétée produit toujours le même état final. `state: present` l'est ; `echo ... > fichier` via `shell` ne l'est pas. Le cours recommande : nommer chaque tâche, qualifier les modules (`ansible.builtin.apt`), préférer les modules dédiés à `command/shell`, découper en rôles, mettre les secrets dans Vault, valider avec `--syntax-check` puis `--check --diff`. **copy vs template** : contenu identique partout → `copy` ; contenu variable selon l'hôte → `template` (Jinja2). **command / shell / raw** : `command` exécute sans shell, `shell` autorise les tubes, `raw` contourne Python ; aucun n'est idempotent par nature, d'où l'usage de `creates`, `removes` ou `changed_when`.

2.5 Variables, facts, templates et handlers

Les variables proviennent de la ligne de commande, du playbook, de l'inventaire, de `group_vars/` `host_vars`, des rôles (`defaults/`, `vars/`) ou de `register/set_fact`. La **précédence** va des `defaults` d'un rôle (la plus faible) aux variables `-e` en ligne de commande (la plus forte).

Les **facts** sont découverts automatiquement (`gather_facts`). **Jinja2** fournit `{{ variable }}`, `{% if %}`, `{% for %}` et des filtres (`default`, `join`, `to_json`, `regex_replace`). Un **handler** est une tâche notifiée, exécutée une seule fois en fin de play, uniquement si la tâche appelante retourne `changed`.

2.6 Rôles, Vault, tags et erreurs

Un **rôle** regroupe tâches, variables, templates, fichiers et handlers d'une responsabilité précise, selon l'arborescence `tasks/`, `handlers/`, `templates/`, `files/`, `vars/`, `defaults/`, `meta/`. Un playbook orchestrateur (`site.yml`) les compose, éventuellement conditionnés par `group_names`.

Ansible Vault chiffre en AES256 fichiers ou variables sensibles (`create`, `edit`, `encrypt`, `encrypt_string`, `view`). La convention du cours : la valeur chiffrée porte un préfixe `vault_` et une variable en clair y fait référence.

Les **tags** permettent une exécution sélective (`--tags`, `--skip-tags`, `--list-tags`). La **gestion des erreurs** repose sur `block/rescue/always`, équivalent d'un `try/catch/finally`.

2.7 Collections, performance et sécurité

Depuis Ansible 2.10, le cœur `ansible-core` ne contient que `ansible.builtin.*`; le reste est distribué en **collections** (`community.general`, `amazon.aws`, `community.docker...`), déclarées dans `requirements.yml`.

Performance : `forks`, `strategy: free, pipelining, async/poll`. **Sécurité** : clés SSH plutôt que mots de passe, moindre privilège, Vault pour tous les secrets, accès Git restreint, audit des collections tierces, `--limit` lors des tests.

2.8 Étude de cas : architecture trois tiers

Le cours conclut sur le patron le plus courant en production : inventaire structuré en groupes (`loadbalancers`, `webserver`s, `appserver`s, `dbserver`s), un rôle par responsabilité, et un playbook orchestrateur multi-plays respectant l'ordre métier (base de données, puis application, puis frontal).

Partie II — L'énoncé du projet 3 et les attentes

3 Énoncé et besoin à couvrir

Intitulé. « Développer une application web Flask permettant aux utilisateurs de louer et gérer via SSH des instances (conteneurs) hautement disponibles depuis un dashboard. »

Besoin auquel le projet répond. « Répondre au besoin d'un utilisateur de créer un compte, louer une VM/conteneur selon la distribution et la durée voulues, y accéder en SSH, avec haute disponibilité en cas de crash. »

Description détaillée. L'énoncé précise six exigences :

1. une application web Flask permettant à un visiteur de se connecter à son compte ou d'en créer un ;
2. une redirection vers un *Dashboard* ;
3. depuis lequel il peut louer un accès à des machines virtuelles (conteneurs), **de différentes distributions, pour une durée qu'il choisit** ;
4. il accède à ses instances via cette même page ;
5. **un accès SSH lui est donné lors de sa location** ;
6. le service doit être **résilient** et fournir une **haute disponibilité en cas de crash ou de perte d'accès** à ses instances.

Quatre catégories de tâches sont imposées : création de l'API web ; création de la partie Ansible (playbook, etc.) ; création de la base de données (locations, comptes, etc.) ; déploiement des VM (docker compose, etc.).

4 Modalités de validation

Élément	Attendu
Rapport écrit	PDF, 8 pages maximum , déposé sur Celene avant la défense. Cinq sections : introduction (objectif et portée) ; aperçu de l'architecture (un paragraphe par décision architecturale) ; fonctionnalités (liste complète, captures d'écran, bonne explication) ; difficultés ; orientations futures.
Défense finale	Discours d'introduction (contexte, objectifs, choix) puis démonstration pratique : provisionnement et déploiement automatisés en direct . Deux séances par groupe, les 12 et 13 octobre.
Organisation	Groupes de 4 à 5 personnes, méthode de suivi par galon : réunion de lancement, deux points d'avancement, présentation finale. Gestion des tâches suggérée par <i>user stories</i> et Trello.

Table 3 – Modalités de validation du cours.

Remarque. Le projet a été réalisé seul, alors qu'il est prévu pour un groupe de quatre à cinq personnes. Les quatre catégories de tâches ont donc été menées par le même auteur.

Partie III — Ce qui a été réalisé et analyse de conformité

5 Conformité aux exigences de l'énoncé

Exigence de l'énoncé	Ce qui a été réalisé	Verdict
Application web Flask, connexion ou création de compte	Flask servi par Gunicorn derrière nginx. Inscription et connexion, mots de passe hachés (PBKDF2 salé, Werkzeug), politique de mot de passe, verrouillage après cinq échecs, sessions signées.	Conforme
Redirection vers un <i>Dashboard</i>	Redirection automatique après authentification ; console listant les machines, leur état Docker réel, le temps restant et les actions.	Conforme
Louer des conteneurs de différentes distributions	Trois distributions : Ubuntu 22.04, Debian 12, Alpine 3.20 — chacune en deux variantes (terminal, bureau graphique), soit six images construites depuis trois Dockerfile .	Conforme
Pour une durée choisie	Durées prédéfinies (5, 10, 15, 30, 60 min) ou libre de 1 à 120 min ; prolongation possible ; compte à rebours affiché ; destruction automatique à l'expiration.	Conforme
Accès aux instances depuis la même page	Le tableau de bord liste les machines actives avec commande SSH, boutons <i>Terminal</i> et <i>Bureau</i> , prolongation, régénération du mot de passe et fin de location.	Conforme
Accès SSH donné lors de la location	Commande complète affichée (<code>ssh root@<hôte> -p <port></code>), port aléatoire vérifié libre, mot de passe root tiré au sort affiché une fois, ou clé publique SSH de l'utilisateur injectée dans la machine.	Conforme
Résilience et haute disponibilité en cas de crash	Trois niveaux : supervisord en PID 1 relance les services internes ; <code>--restart=always</code> relance le conteneur ; live-restore conserve les conteneurs pendant un redémarrage du démon. Un <i>watchdog</i> systemd relance le site, le Faucheur et nginx.	Au-delà
Tâche 1 — API web	Application Flask complète : 11 routes, coffre à secrets, ordonnanceur, interface console (HTML + CSS + JavaScript sans framework).	Conforme
Tâche 2 — Partie Ansible	Playbook de 4 plays et 8 rôles, inventaire à groupes, group_vars , templates Jinja2, handlers, tags, Vault, collections. Idempotence vérifiée (<code>changed=0</code>).	Au-delà
Tâche 3 — Base de données	SQLite en mode WAL : tables users , instances (locations) et login_attempts , avec index et migrations automatiques.	Conforme

Exigence de l'énoncé	Ce qui a été réalisé	Verdict
Tâche 4 — Déploiement des VM (<code>docker compose</code>)	Les VM sont provisionnées par Vagrant. Les locations sont créées par <code>docker run</code> depuis Flask (<code>subprocess</code>), seule façon d'obtenir un port et une image choisis à la volée. Un fichier <code>docker-compose.yml</code> complète le dispositif : il déclare la construction des six images (profil <code>build</code>) et une machine de démonstration reproductible (profil <code>demo</code>).	Conforme

6 Couverture des notions du cours

Notion du cours	Mise en œuvre dans le projet	Verdict
Cours 1 — Virtualisation, conteneurs, Vagrant		
Virtualisation par VM	Trois VM Ubuntu 22.04 décrites dans le <code>Vagrantfile</code> .	Conforme
Virtualisation par conteneurs	Six images Docker, isolation par namespaces et cgroups.	Conforme
Dockerfile	Trois Dockerfile utilisant FROM, ARG, ENV, RUN, COPY, EXPOSE, CMD, avec construction conditionnelle (<code>--build-arg DESKTOP=1</code>).	Conforme
Commandes Docker	<code>build</code> , <code>run</code> , <code>ps</code> , <code>inspect</code> , <code>exec</code> , <code>rm</code> , images utilisées par l'application et le Faucheur.	Conforme
Docker Compose	<code>1_docker/docker-compose.yml</code> : sept services, deux profils (<code>build</code> , <code>demo</code>), limites et réservations déclaratives. Les locations restent en <code>docker run</code> , leurs ports étant tirés au hasard.	Conforme
Limites de ressources	<code>--memory</code> , <code>--cpus</code> , <code>--pids-limit</code> , <code>--shm-size</code> appliqués à chaque conteneur.	Conforme
Réservation de ressources	<code>--memory-reservation</code> accompagne chaque limite : 128 Mio garantis pour 256 Mio en mode terminal, 512 Mio pour 1 Gio en mode bureau.	Conforme
Supervision (<code>docker stats</code>)	Le <i>watchdog</i> relève chaque minute la consommation CPU et mémoire de chaque machine louée, sur chaque nœud, et journalise une alerte au-delà de 90 % de CPU.	Conforme
Mise à l'échelle	Horizontale au niveau du parc : ajout d'un worker dans l'inventaire, répartition automatique.	Conforme
Vagrant : box, provider, provisioner	Box <code>bento/ubuntu-22.04</code> , quatre providers (VMware, Parallels, QEMU, VirtualBox), provisioner Shell.	Conforme
Vagrant multi-machines	Trois machines définies (<code>controller</code> , <code>worker1</code> , <code>worker2</code>).	Conforme
Réseau privé et IP fixe	<code>private_network</code> en 192.168.56.10 / .11 / .12.	Conforme
Configuration des ressources VM	<code>memory</code> et <code>cpus</code> définis par machine et par provider.	Conforme
Provisioner Ansible déclaratif	Le <code>Vagrantfile</code> enchaîne un provisioner Shell (Python) puis le provisioner <code>ansible</code> , déclenché après la dernière VM : <code>vagrant up</code> suffit à obtenir la plateforme complète.	Conforme

Notion du cours	Mise en œuvre dans le projet	Verdict
.vagrant/ non versionné	Exclu par .gitignore.	Conforme
Cours 2 — Ansible		
Inventaire, groupes, :children	[controllers], [workers], [insaccloud:children].	Conforme
group_vars	Trois répertoires : inaccloud, controllers, workers.	Conforme
ansible.cfg	Inventaire, roles_path, forks, host_key_checking, élévation de privilèges.	Conforme
Commandes ad hoc	ansible inaccloud -m ping utilisé pour valider la connectivité.	Conforme
Playbooks multi-plays	Quatre plays successifs : durcissement, clé du contrôleur, workers, contrôleur.	Conforme
Modules dédiés	21 modules distincts, tous qualifiés (ansible.builtin.*, ansible.posix.*, community.general.*).	Au-delà
Idempotence	Second passage : changed=0 sur les trois nœuds (mesuré).	Conforme
Variables et facts	defaults, group_vars, register, set_fact, facts ansible_architecture et ansible_distribution_release.	Conforme
Templates Jinja2	Cinq templates avec conditions, boucles et filtres (to_nice_json, join, default).	Conforme
Handlers	Six handlers (daemon-reload, redémarrage des services, rechargement de nginx et sshd).	Conforme
Conditions et boucles	15 when, 14 loop avec loop_control et index_var.	Conforme
Rôles	Huit rôles avec defaults, tasks, handlers, templates.	Conforme
meta/main.yml (dépendances)	Les huit rôles déclarent leurs métadonnées et leurs dépendances (worker dépend de docker, reverse_proxy et webapp de tls_cert...). Ansible les résout et les dédouble.	Conforme
Ansible Vault	vault.yml chiffré AES256, convention vault_* exactement comme le cours, deux secrets.	Conforme
Tags	11 tags (security, docker, tls, workers, images, proxy, webapp...).	Conforme
Gestion des erreurs block/rescue	La construction des images, étape la plus fragile, est encadrée par block/rescue/always : le rescue produit un diagnostic (image en cause, message de docker build, erreurs du démon, espace disque), le always inventorie les images.	Conforme
Collections	requirements.yml : community.general et ansible.posix.	Conforme
Inventaire dynamique	inventaire_vagrant.py interroge vagrant ssh-config et produit adresses, ports, clés et groupes au format JSON. Il rend l'inventaire indépendant du provider, dont le nom est inscrit en dur dans la variante statique.	Conforme

Notion du cours	Mise en œuvre dans le projet	Verdict
Performance (forks, pipelining, async)	forks, pipelining et multiplexage SSH (ControlMaster, ControlPersist) activés. async reste inutile : aucune tâche ne dépasse la durée d'une connexion.	Conforme
Sécurité (clés SSH, Vault, --limit)	Clés SSH partout, Vault pour les secrets, --limit utilisé en test.	Conforme
Étude de cas trois tiers	Même patron appliqué : inventaire en groupes, un rôle par responsabilité, playbook orchestrateur respectant l'ordre métier.	Conforme
Cours 3 — DevSecOps (notions abordées en cours)		
Gestion des secrets	Ansible Vault, chiffrement Fernet en base, fichier d'environnement 0600, secret effacé de l'environnement du PID 1.	Au-delà
Sécurité des conteneurs	--cap-drop ALL et capacités minimales, no-new-privileges, limites PID et CPU, TLS jusqu'aux machines.	Au-delà
Pipeline CI/CD, SAST, SCA, DAST	.github/workflows/ci.yml : huit travaux à chaque envoi — lint (ruff, ansible-lint, hadolint), 67 tests unitaires et d'intégration, SAST (bandit), SCA (pip-audit), fuite de secrets (gitleaks), DAST (18 contrôles sur l'application en fonctionnement, puis nuclei), scan de l'image Docker (Trivy).	Au-delà
Kubernetes	Hors périmètre du projet 3.	—

7 Éléments réalisés au-delà de l'énoncé

- **Architecture distribuée** contrôleur + deux workers avec ordonnancement sur le nœud le moins chargé, alors que l'énoncé n'imposait qu'un hôte.
- **Deux modes d'accès supplémentaires** : terminal web et bureau graphique complet dans le navigateur, en plus du SSH demandé.
- **Conservation de l'état** : live-restore et relance automatique, vérifiés par redémarrage complet d'un worker.
- **Watchdog** systemd d'auto-réparation du service.
- **Durcissement applicatif** : CSRF, anti-force-brute, CSP stricte sans ressource externe, HTTPS de bout en bout, coffre à secrets chiffré.
- **Chaîne de démonstration** : vidéo du parcours complet et captures d'écran versionnées.
- **Chaîne d'intégration continue** de huit travaux couvrant le cycle DevSecOps complet : lint, tests, SAST, SCA, recherche de secrets, DAST et scan d'image.
- **67 tests automatisés** (sécurité, orchestration Docker, Faucheur) exécutés sans Docker ni réseau, donc rejouables sur n'importe quel poste.
- **Inventaire dynamique** interrogeant Vagrant, ce qui rend le déploiement indépendant du *provider* de virtualisation.

8 Conformité au document « PROJET 3 » (17 fonctionnalités)

Le document détaillé remis par l'enseignante décompose le projet en dix-sept fonctionnalités, chacune assortie d'une liste de points à couvrir. Le tableau suivant reprend cette décomposition telle quelle.

#	Fonctionnalité	Réalisation	
1	Préparation et environnement	Arborescence en trois répertoires, dépôt Git, <code>.gitignore</code> , <code>README.md</code> , <code>.env.example</code> et stratégie de branches décrite dans <code>CONTRIBUTING.md</code> .	Conforme
2	API web Flask	Inscription, connexion, déconnexion, sessions, hachage <code>script</code> , tableau de bord protégé. Les onze points d'entrée demandés existent avec leurs chemins exacts, de <code>GET /health</code> à <code>POST /workers/heartbeat</code> . Application découpée en huit modules et trois profils de configuration.	Conforme
3	Base de données	Les cinq entités demandées en SQLAlchemy avec leurs relations : <code>User</code> (dont <code>email</code>), <code>Distribution</code> , <code>Worker</code> , <code>Instance</code> , <code>Rental</code> . Migrations Alembic réversibles, jeu de données de démonstration, contraintes d'intégrité vérifiées par des tests.	Conforme
4	Conteneurisation Docker	<code>Dockerfile</code> de l'application en deux étapes, compte sans privilège (<code>uid 10001</code>), sonde de santé. <code>docker-compose.yml</code> unique : base de données, application, Faucheur, les six images louables et une machine de démonstration. <code>docker compose up -d</code> donne une plateforme fonctionnelle.	Conforme
5	Infrastructure Vagrant	Contrôleur et trois workers, adresses fixes sur réseau privé, parc paramétrable par <code>WORKERS=n</code> . Provisionnement Ansible enchaîné automatiquement.	Conforme
6	Automatisation Ansible	<code>hosts.ini</code> avec les groupes demandés, quatre playbooks (<code>site</code> , <code>docker</code> , <code>worker</code> , <code>deploy</code>), neuf rôles, idempotence mesurée à <code>changed=0</code> sur quatre nœuds. L'agent est déployé par son propre rôle.	Conforme
7	Agent et enregistrement dynamique	<code>worker_agent.py</code> inscrit le nœud puis envoie un battement de cœur toutes les trente secondes. Il relève cœurs, mémoire et état de Docker, et n'utilise que la bibliothèque standard : rien à installer sur un worker. États <code>AVAILABLE</code> , <code>BUSY</code> , <code>OFFLINE</code> .	Conforme
8	Gestionnaire de ressources	Sélection du nœud joignable le moins chargé, capacité déduite de la mémoire, points d'entrée <code>GET /workers</code> et <code>GET /workers/{id}</code> .	Conforme
9	Service de location	Formulaire, choix de la distribution et de la durée, validation, confirmation. Côté serveur : vérification du quota, choix du nœud, création de l'instance et de la location en une transaction.	Conforme
10	Déploiement des instances	Nom unique, image choisie, limites CPU et mémoire avec réservation, port SSH publié, identifiants enregistrés. Destruction : arrêt, suppression, location passée à <code>EXPIRED</code> .	Conforme
11	Accès SSH	SSH configuré dans les trois images, mot de passe tiré au sort affiché une seule fois ou clé publique de l'utilisateur, commande affichée sur le tableau de bord, accès révoqué à l'expiration puisque la machine est détruite.	Conforme

#	Fonctionnalité	Réalisation	
12	Gestion de l'expiration	Le Faucheur relit la base toutes les dix secondes, détruit les machines échues et met la location à jour. Mesuré : destruction 49s après l'expiration d'une location d'une minute.	Conforme
13	Haute disponibilité	Battement de cœur, délai d'expiration, passage OFFLINE, recherche d'un nœud de repli et recréation du conteneur. Le scénario du document (vagrant halt worker1 , l'instance repart sur worker2) est vérifié sur un cluster réel : la location n'est pas touchée, l'utilisateur garde sa durée.	Conforme
14	Sécurité DevSecOps	Validation des entrées, hachage, secrets au Vault, chiffrement Fernet en base, image minimale sans secret, compte sans privilège, scan Trivy, Vault Ansible, privilèges limités, cloisonnement entre comptes vérifié par un test.	Au-delà
15	CI/CD	Onze travaux : lint, tests, sécurité, build, scan d'image, SBOM, registre, déploiement. La chaîne publie des images étiquetées version, commit et latest , et refuse toute image portant une faille critique.	Au-delà
16	Tests d'intégration	Les six scénarios du document dans tests/integration.sh : 19 contrôles sur le cluster Vagrant, 18 sur la pile compose. Les tests 1 à 5 sont rejoués à chaque envoi par l'intégration continue.	Conforme
17	Documentation et soutenance	Schémas d'architecture, de réseau, de cycle de vie, du modèle de données et de la chaîne CI/CD. Documentation de l'API, de la base, de Docker, de Vagrant et d'Ansible. Procédures d'installation, de déploiement et de test, scénario de panne, limites et perspectives.	Conforme

Table 6 – Les dix-sept fonctionnalités du document PROJET 3.

Les dix-sept fonctionnalités sont couvertes, deux d'entre elles au-delà de ce qui est demandé.

9 Écarts relevés et travaux réalisés

La relecture intégrale des quatre supports de cours avait mis en évidence huit écarts. Tous ont été traités, et chaque correction a été vérifiée sur le cluster de trois nœuds avant d'être retenue.

Vérifications conduites sur le cluster

Un cluster de trois nœuds (un contrôleur, deux workers) a été remonté pour éprouver l'ensemble :

- **Déploiement complet** : **failed=0** sur les trois nœuds.
- **Idempotence** : second passage à **changed=0** sur les trois nœuds, après toutes les modifications.
- **Parcours utilisateur** : inscription, location d'une machine Alpine, affichage des accès ; le conteneur est bien créé sur le worker le moins chargé.

#	Écart relevé	Traitement et vérification	État
1	Docker Compose absent	1_docker/docker-compose.yml : six services de construction (profil build) et une machine de démonstration (profil demo). Vérifié par <code>docker compose -profile demo up</code> , connexion SSH réussie.	Conforme
2	Pas de réservation mémoire	--memory-reservation ajouté à chaque lancement. <code>docker inspect</code> d'une machine louée : limite 256 Mio, réservation 128 Mio.	Conforme
3	Provisioner Ansible absent du Vagrantfile	Provisioner ansible déclenché après la dernière VM, avec <code>ask_vault_pass</code> .	Conforme
4	block/rescue/always jamais employé	Encadre la construction des images. Vérifié en forçant un échec : <code>rescued=1</code> , diagnostic donnant l'image, le message de <code>docker build</code> , les erreurs du démon et l'espace disque.	Conforme
5	meta/main.yml absent	Les huit rôles déclarent métadonnées et dépendances ; Ansible les résout et les dédoublonne à l'exécution.	Conforme
6	Aucune métrique de consommation	Le <i>watchdog</i> relève <code>docker stats</code> sur chaque nœud toutes les minutes et alerte au-delà de 90 % de CPU.	Conforme
7	Aucun inventaire dynamique	<code>inventaire_vagrant.py</code> interroge <code>vagrant ssh-config</code> . Corrige au passage un défaut réel : le chemin de clé codé en dur sur <code>virtualbox</code> .	Conforme
8	Aucune chaîne CI/CD	Huit travaux GitHub Actions ; chaque commande a été rejouée localement avant d'être inscrite dans le fichier.	Au-delà

Table 7 – Les huit écarts relevés à la relecture du cours, et leur traitement.

- **Durcissement effectif** : `docker inspect` confirme `restart=always`, limite et réservation mémoire, `pids-limit`, `cap-drop ALL` et `no-new-privileges`.
- **Conservation de l'état** : après redémarrage du démon Docker *puis* du nœud entier, la machine louée et le travail de l'utilisateur sont intacts.
- **Faucheur** : une location d'une minute est détruite 49s après son expiration, le conteneur retiré du worker et l'instance marquée `expired`.
- **Analyse dynamique** : 18 contrôles sur l'application en fonctionnement, tous réussis ; `nuclei` ne signale que le certificat auto-signé, choix assumé et documenté.

Ce qu'il reste à préparer

Aucun écart technique ne subsiste. Restent deux points de préparation, qui ne relèvent pas du code : répéter la démonstration `vagrant up` suivie du playbook pour en mesurer la durée réelle, et préparer le discours d'introduction attendu en début de soutenance.

Conclusion

Les six exigences fonctionnelles de l'énoncé sont satisfaites, plusieurs au-delà de ce qui était demandé : haute disponibilité par cluster de trois nœuds, trois distributions en deux modes d'accès, conservation de l'état vérifiée par redémarrage complet. Les quatre catégories de tâches imposées — interface web, Ansible, base de données, déploiement des machines — sont toutes couvertes, y compris `docker compose`, seule réserve de la première version de ce dossier.

Sur les notions du cours, la couverture est désormais complète : les cinquante critères relevés dans

les quatre supports sont satisfaits, six d'entre eux au-delà de ce qui était enseigné. Les huit écarts identifiés lors de la relecture ont tous été comblés, et chaque correction a été éprouvée sur un cluster réel plutôt que simplement déclarée.

Le projet dépasse enfin le périmètre du cours sur un point : une chaîne d'intégration continue rejoue à chaque envoi l'analyse statique, les 67 tests, l'audit des dépendances, la recherche de secrets, l'analyse dynamique de l'application et le scan de l'image Docker. C'est la garantie que les propriétés décrites dans ce dossier restent vraies, et non qu'elles l'ont été une fois.